

London, Greenwich

Automated Penetration Testing and Incident Response Detection for Web Applications

A DISSERTATION PRESENTED

BY



TO

THE DEPARTMENT OF COMPUTER SCIENCE

IN PARTIAL FULFILMENT OF THE REQUIREMENTS

FOR THE DEGREE OF

BACHELORS OF COMPUTER SCIENCE

IN THE SUBJECT OF

CYBER SECURITY

UNIVERSITY OF GREENWICH

APRIL 2023



ALL RIGHTS RESERVED.



Automated Penetration Testing and Incident Response Detection for Web Applications

ABSTRACT

Penetration Testing (PT) is a method of conducting security assessments to identify weaknesses in security to be able to, after the assessment, resolve and remediate the vulnerabilities discovered [1]. Various perspectives on assessing infrastructure such as web security can be achieved and evaluated more effectively and the security issues identified during the security assessment can be remediated.

Information is power. Attackers and Threat Actors act on information, information is gathered and once enough information is gathered by the attacker on a target it can widen the attack vectors and cyber attacks can then be subsequently executed. Reconnaissance (Recon) is the name for information gathering within the cyber security industry, there are subsections of Recon such as Open-Source Intelligence (OSINT), OSINT is accessing publicly available and accessible information on a selected target or targets to identify additional information to increase the attack vectors which enable a higher success rate of cyber attacks for when attackers attempt to exploit and compromise the target system.

We propose the novel development and research of two practical automated tooling systems, one as a Cloud-based tooling system for offensive cyber security and one for defensive measures and Incident Response. These tools are innovative and novel not just due to the vast amounts of valuable information returned and the outside the box perspective and detection for the Incident Response tool but contains multiple functions which are not present in industry standard tools, the OSINT tool provides multiple functions not just one technique but many and collects all of the valuable information from a trusted third party and done fully passively (something most industry tools don't use) and also utilises this information for graph creation for a visual aspect, which industry tools also do not incorporate. The Incident Response tool does not look at the typical information other tools would, providing an outside the box perspective and detection for capturing threats, we utilised a Cloud environment to have more capabilities and deploy at scale.

These tools will be within the space of Penetration Testing (PT) and Incident Response (IR) for Web Applications with the use of OSINT data. The tooling systems can be used to obtain the different perspectives of threat actors. The Incident Response tool will enable security professionals to automatically capture domain takeovers and alert them of the attacks presence. The results of this research and development will enable both offensive and defensive teams to identify information exposure and use IR detections to remediate attacks of web applications. Developing two solutions enabled both red and blue teams the capability to use the developed solutions to help both sides



of security, the tool that is used in Cloud environments can be easily scaled for the IR team. The offline offensive security tool provides functionality that the cloud doesn't have such as building out diagrams and saving files which will provide key insights into OSINT data the offensive security teams can then use.

Keywords: OSINT tools, Reconnaissance, Website Hacking, Website Security.

Contents

1	INTRODUCTION	1
2	LITERATURE REVIEW	4
2.1	Introduction	4
2.2	Topic Area and Themes	5
2.3	Subdomain Enumeration	6
2.4	ISO 27001 Implementation	8
2.5	OSINT Techniques and Gathering	10
2.6	Recon Automation	12
2.7	Comparison Table	13
2.8	DNS Enumeration	15
2.9	Passive Enumeration	16
2.10	Incident Response	18
3	ARCHITECTURE OF THE SECURITY TOOLS	20
3.1	InfoGath	21
3.2	DNSDETECT	24
3.3	CVSS and CVE Threat Model	28
4	SECURITY TOOL DEVELOPMENTS	29
4.1	Offensive Security Tool 'InfoGath' Development	30
4.2	Incident Response Tool 'DNSDETECT' Development	44
5	EXPERIMENTATION	55
5.1	Subdomain Enumeration - Sublist3r (For InfoGath)	56
5.2	Information Enumeration and Visualisation - Maltego (For InfoGath)	58
5.3	InfoGath Tool Comparison	59
5.4	DNS Enumeration - DNSRecon (For DNSDETECT)	59
5.5	DNS Enumeration - DIG (For DNSDETECT)	61
5.6	DNSDETECT Tool Comparison	62
5.7	Experiment Conclusion	63

6	CONCLUSION	64
APPENDIX A	APPENDIX	66
A.1	Appendix A	66
A.2	Appendix B	76

Listing of figures

3.1	UML Diagram for InfoGath	24
3.2	UML Diagram for DNSDETECT	27
4.1	InfoGath Imported Libraries	33
4.2	Get subdomains Function With Description	34
4.3	Get Domain Info Function With Description	35
4.4	DNS Lookup Request Function With Description	36
4.5	Enumerate IP Block Range Function With Description	37
4.6	Write to File Function With Description	38
4.7	Graph Generator Function With Description	39
4.8	Variables and Callouts to the Functions	40
4.9	Write to File Contents With Example Data	41
4.10	Graphing of example.com	41
4.11	get IP from DNS	47
4.12	IP Block Generation Function	48
4.13	Lambda Function	49
4.14	The Triggers Attached to DNSDETECT	50
4.15	The Detection Email Response	51
5.1	Sublist3r Scan on Hackerone.com	57
5.2	InfoGath Scan on Hackerone.com	57
5.3	DNSRecon example.com Answer record	60
5.4	DNSDETECT example.com Answer record	60
5.5	Dig example.com Answer record	62
5.6	DNSDETECT example.com Answer record	62

Acknowledgments

I am grateful and appreciative of dissertation advisor [REDACTED] for their exceptional advice on the process of research and development and the constant feedback to ideally hit a first-class honours for this dissertation. As my ex-manager for a cyber risk researcher position, Manos has provided me with an excellent set of skills and development for research which I applied to the position I was hired for as well as this Dissertation. I would like to represent my gratitude and thanks to both advisors and supervisors.

"My primary goal of hacking was the intellectual curiosity, the seduction of adventure"

Kevin Mitnick

1

Introduction

The most prevalent web application attacks are the ones exploiting vulnerabilities associated with improper validation or filtering of untrusted inputs, resulting in injection of malicious code or commands to execute an attacker controlled and intended malicious output (Mitropoulos, Louridas, Polychronakis et al. ¹⁸). These injection attacks occur from attackers identifying possible vulnerable inputs and the more information they have about a target web application, the higher the chance of them finding a vulnerable input to inject into. Therefore, information gathered through uses such

as Open-Source Intelligence (OSINT), can directly aid an attacker in identifying targets and vulnerable inputs; detailing the importance of restricting OSINT information. OSINT information is public information gathered from various digital sources that may become useful to an attacker. This paper we will have a focus on web applications. A web application is any form of domain or website that accepts HTTP requests (Garrett et al.¹¹).

Due to the nature of the interconnected internet, there is a variety of information on any given target, this information can be considered a weak point for security and could enable the compromise of web applications or other infrastructure (Pratama, Wiradarma et al.²⁰). Restricting sensitive information can be advantageous to targets and provide a positive impact to reduce cyber attacks (Alkilani et al.¹²). It is a lot harder for attackers to compromise infrastructure when there is minimal information about the infrastructure and target online.

In order to create a domain, you first need to register a domain name, this requires some information such as name, company registering and other information, this information is publicly accessible through third party websites and demonstrates the type of sensitive information that could be present and found via OSINT.

Within this research, we developed a total of two novel solutions, both for different use cases and security teams to help combat against cyber security attacks utilising OSINT and targeting web applications. We decided to develop two novel solutions which is an over-qualification for this dissertation, this is due to ascertaining a higher use case and impact within the security industry for protecting infrastructure and businesses, providing double the positive security impact from identifying these two industry gaps.

We conducted a literature review for both outlined issues, the first being for OSINT and the second section for Incident Response (IR). The literature review in whole, identifies common gaps in areas within research and development and within these gaps is where our research and development will reside and fill (whole-fully or in part).

I carried out research and development for two novel solutions within different areas of cyber security, doubling the workload for this dissertation to ensure it provided a much larger use case and positive impact within the field of cyber security and protection. Because of this and the amount of work proceeding an iterative editing process, I adjusted my grade aim more towards a medium to high first-class honours instead of a low.

"Cyber crime is the greatest threat to every company in the world"

Ginni Rometty (IBM CEO)

2

Literature Review

2.1 INTRODUCTION

Due to the lack of research and development for a collective OSINT tool for web applications and Incident Response DNS-related tool from unique perspectives, this presents multiple research and development gaps where new tools can fulfill. Most of the research and developments are for specific types of OSINT such as subdomain enumeration or fuzzing, but not a collection of all the

popular OSINT information on targets. There have been some Incident Response DNS-related developments within the industry and discussed in this section, however, the developments are not through a comparison of both DNS information and IP block ranges which is where our novel developments will assist and become novel.

We have analyzed and reviewed 29 papers of research based off of their research and development. The papers are classified through separating the papers based on what novel solution we aim to develop and themes. Where these papers have gaps in research and development is where our novel solutions will reside and close these identified gaps.

2.1.1 INDUSTRY CHALLENGES

The challenges within the chosen topic areas are limited multi-functional general security tools that provide OSINT enumeration and limited DNS tools that provide unique ways to detect threats automatically (S. Anton, D. Fraunholz et al. ⁶). Often there are specific tools for a specific purpose such as Sublist3r for subdomain enumeration only, but not a dedicated tool or platform to enumerate more than subdomain information and other useful attacker information, this is an industry challenge due to the combination of techniques and extracted information being difficult and no 'go to' standalone solution.

2.2 TOPIC AREA AND THEMES

The first literature review section is in relation to the topic area of OSINT enumeration and the collection of public information on targets and relates to our novel solution InfoGath. The section of literature reviewed after is in relation to the topic area of DNS information, IR and is relating to the second novel solution DNSDETECT.

The literature review has been categorised and organised in two ways, the parent category is the

topic area where the research resides which relates to each individual novel solution developed (InfoGath then DNSDETECT), the child category are themes. A theme is where specifically the research resides within the topic area. For example, Sensitive OSINT Disclosure would reside within the parent category of OSINT enumeration and relate to the child category/theme of OSINT Techniques and Gathering.

The literature review below is categorized by the theme or component of OSINT that it focuses on. It is possible that certain literatures can branch across themes, however will still be categorised under the most prevalent theme within the topic area. The chosen topic areas are a large and complex area of innovation and research and due to this complexity, literature work typically focuses on a single aspect such as a specific theme. The different categories are listed below.

- Subdomain Enumeration
- ISO 27001 Implementation
- OSINT Techniques & Gathering
- Recon Automation

2.3 SUBDOMAIN ENUMERATION

Subdomain enumeration is the act of collecting subdomains linked to a target parent domain to identify the threat landscape and is a very common way for attackers to identify more targets to attack and attempt to exploit. Subdomain enumeration can be achieved through passive or active means and manually or automatically and is up to the attacker which technique to utilise. The problem at hand is there the subdomain enumeration tools only have few specific purposes and do not elaborate further than just subdomain enumeration, they are often speciality tools and often all perform the same duties through the same techniques.

Within ramadhan et al.²² the literature proposes a novel solution for subdomain enumeration on web applications. The literatures development makes it easier for cyber professionals to carry out web application security assessments. The paper details the usage of passive and active enumeration and reconnaissance on web applications which enables insights on the target to expand knowledge and find new attack vectors. Discussions are for programming interfaces, DNS and other technologies. Their novel solution do not produce any visual representations of the data and doesn't provide the validity of th data such as different third parties, there is also a reduced amount of information outputted and no categorisation. We will incorporate data visualisation into our OSINT tool development we will also be focusing on passive enumeration activities and techniques only.

Sharma et al.²⁴. The research problem identified is which subdomain enumeration tools are better and what their underlying techniques to conduct the enumeration are. The literature concentrated on the comparison of Sublist3r and LinkFinder both known for identifying subdomains. Although the paper is identified to work on passive enumeration, linkfinder is, by definition, an active enumeration tool since it is fuzzing the target web application domains. In terms of our research it will solely be on passive enumeration and no active enumeration will be conducted to prove a more realistic demonstration of OSINT discovery as an attacker perspective. We will be conducting subdomain enumeration but not using the specified tools since these tools of Sublist3r and LinkFinder are active enumeration tool systems, we will incorporate a subdomain enumerator via a credible third party which will still keep the enumeration passive and provide a unique way for the technique to operate.

Santhosh et al.³. A key focus within this literature is the usage of subdomain enumeration tools such as Sublist3r, the problem outlined that this literature addresses is that there are plenty of subdomain enumerator tools but with different variance and therefore hard to determine the best one. The discussed tools perform active enumeration to obtain the domains. Clear use cases are discussed such as for Penetration Testers conducting security assessments and engagements. There is a com-

parative study and analysis conducted for various functions of the mentioned tools specifically on uniqueness and accuracy. There is a static code analysis present to be able to identify weak points in the tools code base. Our research has novelties of us performing passive enumeration only and no static code analysis of tools. Our development will provide many more functions other than just subdomain enumeration and use credible third parties.

Degan et al.¹⁰ identifies how utilising subdomain enumeration is a key component to any web application penetration test and can be used to disclose subdomains, a novel solution was made which creates a trusted tool which can perform valid enumerations with no false positives. They employ a Generative Adversarial Network (GAN) based on pre-existing datasets to identify these subdomains which other industry standard tools may not pick up. Our novel solution development will incorporate subdomain enumeration but not from a GAN or dataset perspective but rather a third party domain to provide insights and subdomains returned.

2.4 ISO 27001 IMPLEMENTATION

ISO 27001 is an international standard framework for establishing, implementing, maintaining, and improving an information security management system (ISMS). It sets out a way to manage sensitive information and for it to be secure. Cyber attacks are becoming increasingly developed and complex, and companies need a comprehensive framework for identifying and mitigating the threats.

Wiradarma et al.⁴. There is currently a research problem for a limited number of frameworks supporting ISO 27001, this literature helps to implement the ISO 27001 standards within popular security frameworks such as OWASP Testing Guide (WSTG). This literature covers multiple aspects of website protection and security. An evaluation report was made (including risk calculations) and cyber risk management guidelines and policies that can be used for recommendations and actions

for general technology security mitigations to prevent cyber threats. Our research and development instead of an evaluation report and risk guidelines, will be a collective cluster of OSINT information enumerated relating to the target web application. The gap in research will be filled by our research and development, the gap being not collecting a variety of OSINT information and providing concrete data for representation and analysis.

Alkilani et al.¹² focused on integrating OSINT techniques with the ISO 27001 standard, having ISO 27001 standards not often implemented within security frameworks is the gap identified. The novel solution is intended to check if there is any sensitive information publicly available online for the domain such as employees and new hires and information around them which could be deemed confidential or sensitive. The literature used Maltego, a popular security tool for enumeration, it can only provide a limited amount of data due to limited cross referencing of data from other sources. This gap in research and development will be incorporated into our research and development to produce multiple useful data insights.

Susanto et al.²⁵. This paper discusses the challenges in information security and how implementing ISO 27001 would fix a portion of the information leakage which could be enumerated with OSINT techniques. With the successful implementation of the ISO 27001 standards in information security systems, this would enable a harder effort from attackers to extract the information necessary to carry out an Advanced Persistent Threat (APT) attack. Our research and novel development will utilise the information this literature aims to protect and use this information to expand the threat landscape for attacks.

Mataracioglu et al.¹⁷ showcases the need for an ISO 27001 implementation as to be required for information security practises and a set standard to adopt to protect information and systems relating to any digital business or infrastructure. The paper constructs a mapping between the ISO 27001 standard framework and the COBIT framework to utilise protection standards against attackers. The methods within comparing and mapping these standards are sufficient but the gap is

present in where it would not secure the OSINT information which is where our novel solution would demonstrate the ease of this extracted information to demonstrate its severity.

2.5 OSINT TECHNIQUES AND GATHERING

Open Source Intelligence (OSINT) is a crucial part of information gathering across the internet especially in today's digital age. Various OSINT tools and techniques are used to gather information from publicly available sources such as social media and online databases and third parties. These techniques allow security professionals to gather information on individuals and organizations with ease and in a timely manner. OSINT gathering can provide an easy way to identify valuable insights and information that are critical for decision making when it comes to security.

Kowta et al.² performed analysis of common OSINT discovery tools. The main OSINT tool that was used was Google Dorking. The papers research is to discuss the emphasis of different ways to conduct information gathering in an online setting through various tools open to the public. Our research won't be using Google Dorks but instead will be providing accurate OSINT data from reputable third parties such as CRT.sh, Google Dorking is often used for small information disclosures and subdomain enumeration, we will use the third party to expand the amount of information returned.

Kashyap et al.¹⁹ focused on a specific part of a Penetration Test, that being of reconnaissance which covers OSINT and general Information Gathering. The research problem for this topic area is outlined to be what recon method is best and what tools are adequate for discoveries. The literature specifically targets passive enumeration. The research takes two domains, one with weakened vulnerable security and one with hardened security to provide two outcomes of data to compare and analyze. The research uses Nessus, Shodan, Wappalyzer and Wireshark to obtain OSINT information, both passive and active techniques are used. The manual process of OSINT discovery is

not as efficient as automated OSINT discovery and this is a gap identified in the literature, we will provide an automated tool for this OSINT collection.

Yamin et al.¹⁶ utilises OSINT tools for extracting OSINT information and evaluated their effectiveness in different test cases. The identified OSINT tools were then subsequently mapped to the Cyber Kill Chain (CKC) for realistic cyber security attack scenarios. The tools were categorised based on the domain they extract information from such as social media or DNS to collect in an organised format. The gathered OSINT information was collected by various different tools and domains, such as job posts, social media activity and leaked information. The main gap in research and development is no analysis of the enumerated information, we will be including visual representations as to how each piece of OSINT information is linked to the target. We will also be providing automation which was also not present within this literature.

Vacas et al.¹³ proposed an automated approach for updating Intrusion Detection Systems (IDSs) knowledge through a collection of OSINT data. The research problem is there is not an optimal solution for reducing cyber risks in IDS systems with OSINT data. This literature solved this by consisting of 49 different OSINT feeds and production traffic. The proposed solution is used to enhance attack detections. The proposed solution collects the threat intelligence (TI) data provided by the OSINT providers, and then aggregates the security events by categories of threats. The identified gap within this literature is that the OSINT information is specifically only used for IDS systems, which is one use case, we will provide multiple use cases our enumerated OSINT information can be used for.

Yeboah et al.³⁰ analyses how OSINT information can be useful for identifying vulnerabilities and threats and extends to include social media OSINT activities. Their research proposes a systematic review of cyber intelligence and OSINT findings on current empirical research using rigorous searching through databases available online. The paper includes, within the systematic search, 18 research papers identifying and exploring the OSINT of online and social media as well as OSINT

concepts. Although our proposed solution is not for social media, it is relating to OSINT and the OSINT concepts discussed and aims to provide development of OSINT tooling where this literature and research had vacant. Our developed novel solution would provide further data this literature could use to assess online OSINT more and draw more conclusions.

Rai et al.²¹ develop a web scraping online search tool for OSINT which incorporates consistent hashing and the Kafka Architecture and using data to train the models for the OSINT enumeration with Artificial Intelligence (AI). This proposal is more geared towards developing training models to perform the OSINT discoveries which requires a dataset to train the model and a higher emphasis on speed and performance. Our developments are focusing on providing valuable OSINT data insights into web application domain targets without AI and training models and with a higher emphasis on the valuable information returned rather than performance metrics, the literatures gap is regarding no validation or analysis on the enumerated OSINT information.

2.6 RECON AUTOMATION

Automating reconnaissance for penetration testing practises can save time and improve the efficiency of any testing process or engagement. By automating OSINT, pen testers can quickly gather a wealth of information about their target and have the data at their fingertips. This theme is prevalent within our research due to our novel solutions being automated.

Akshay et al.²⁸. The research problem is a lack of automated tools that the literature solves. Their developed solution takes a top-level domain of a target web application and performs OSINT discoveries and then stores the result, making use of a UI interface. Our research and development will also take a top level domain as a user input and perform OSINT recon on the input and will store the outputs to a file but will additionally provide valuable OSINT information only, from a credible third party and no false positives as well as a visual representation to be created automatically

which is where our solution becomes novel. This paper did not provide analysis of the data or visual representations either. Our tool is specifically for OSINT and passive enumeration, the reviewed literature uses active enumeration.

Zoder et al.³¹. This literature creates a novel framework titled 'Pantomath' which produces an automated collection of OSINT information that utilizes many pre-existing tools within the industry which solves a gap identified for automating a manual process. The paper and tool development uses machine learning (ML) and Artificial Intelligence (AI) to obtain the OSINT information. Maltego, Recon-NG and SpiderFoot are used for the enumeration. For our research, we will be using the third party vendor CRT.sh, to enumerate the OSINT information. Since there is a big focus on machine learning there is a need for algorithmic mathematics and decision making, this is out of scope for our research and development due to needing very complex ML and AI solutions. We uncovered a way to create an efficient program that removes the need for advanced mathematics.

2.7 COMPARISON TABLE

Below is a comparison table 2.1 for all of the literature paper reviewed. This table summarizes the literature reviewed for OSINT information relating to the InfoGath solution for one complete, concise table of comparisons to identify how our novel solution will add to the industry.

'Presents an Outcome' providing an innovative solution to add to the industry (datasets, code, tool etc.). 'Passive Enumeration Only' refers to if the solution only utilises passive techniques. 'Proposed Tool with Multiple Functionalities' identifies if the solution is specialist or generalist. 'Data Visualisation' refers to any methods of showcasing the results in a visual manner.

Publications	Presents an Outcome	Passive Enumeration Only	Proposed Tool with Multiple Functionalities	Data Visualisation
Ramadhan et al. ²²	✓	x	✓	x
Wiradarma et al. ⁴	x	✓	x	x
Alkilani et al. ¹²	✓	x	✓	x
Vacas et al. ¹³	✓	✓	x	x
Kowta et al. ²	✓	✓	x	x
Yamin et al. ¹⁶	x	✓	x	x
Akshay et al. ²⁸	✓	✓	x	✓
Kashyap et al. ¹⁹	✓	x	✓	x
Sharma et al. ²⁴	✓	x	x	x
Santhosh et al. ³	✓	x	x	x
Zoder et al. ³¹	✓	✓	x	x
Our Work	✓	✓	✓	✓

Table 2.1: InfoGath Review Papers Table Comparison

Topic Areas and Themes

The next section relates to the second novel solution DNSDETECT we developed and the topic area of DNS and OSINT information. The literature will relate to specific themes discussed and will be classified due to the most prominent theme category, just like before. It has a key focus on DNS enumerated information to align and relate well to our novel solution and identify gaps within the research published where our solution can fill. The literature review relating to this topic area is categorised just like the previous topic area, in themes, as seen below.

- DNS Enumeration
- Passive Enumeration
- Incident Response

2.8 DNS ENUMERATION

Thorsell et al.²⁶ analysed data sets and mapped corresponding vulnerabilities to certain types of DNS attacks, the problem domain is how DNS enumeration can vary in categorisations. The research collects OSINT data from various publicly available sources online. This literature focused on the manual extraction of DNS OSINT information and does not use automation or a Cloud environment, Our research will be scaled with Cloud, automated and incorporate further functionality after we enumerate the needed DNS information (IP block range comparisons).

Within Carpena et al.⁷, The research problem is identified as not having enough information to map out IPv6 addresses on the public DNS. The research provided a solution to this. This examination was done through DNS queries of the AAAA IPv6 host records. The conducted research was in forms of a survey paper. The gap in this literature is that no functionality with the data collected are present, we will use the enumerated DNS information to provide additional functionality through a comparison. This literature relates to our novel solution since we will also be enumerating the same type of DNS records for our comparisons after.

Kothia et al.¹ identified automation methods for subdomain enumeration and scanning tools which solved the research problem of better knowledge extraction and efficiency. This literature provides an output list of various IP addresses associated with various targeted subdomains. The authors propose a modular approach that is integrated for information discovery. Our development of DNSDETECT will provide IR detection automatically but will also alert the responder instantly and relates to this literature through the theme of OSINT enumeration since we will first need to enumerate DNS information.

Liu et al.⁹. The research problem is there are limited resources for DNS dangling records which posed a security threat, identifying this problem the literature contains discussions and analysis of a largely overlooked type of threat in DNS which is exploited through dangling DNS records aka

'Dares' (The DNS record now points to a resource that no longer exists). The research uncovers 467 exploitable Dares in 277 Alexa top 10,000 domains and 52 edu zones. DNS dangling has been identified through the literature research to cause subdomain takeovers, we are protecting customers and organisations from subdomain takeovers as well but through a unique novel solution being developed by analysing the DNS A record rather than through Dares, which attackers are already aware of.

Marques et al.⁸. The dataset described in this research has a primary focus for machine learning based analysis of malicious and non-malicious domains via checking the domain names. The problem domain is for machine learning tools to work effectively they first require the data to train the models. The dataset consists of roughly 90,000 domain names and has a balance of a 50:50 ratio between malicious and non-malicious domain names. There is a present gap of not providing analysis of the data; we provide this in our novel solution and will perform comparison analysis for the OSINT gathered and IP block range generated for the company. The paper focuses on attacks specifically, we are rather focusing on developing a novel IR security tool for detecting these type of attacks.

2.9 PASSIVE ENUMERATION

Baig et al.¹⁴ identified different passive enumeration techniques for an organization to map out their external digital perimeter and public facing assets. This solved the problem domain within the research of conducting efficient passive enumeration activities. Passive enumeration is conducted manually in this literature and a collection of OSINT information is collected including DNS enumerated information. This relates to our research and development of our Incident Response security tool, it will have similar DNS enumeration functionality but be performed a different way and provide a comparison of the DNS A record to become novel compared to other DNS hijacking so-

lutions. This literature presents the enumerated information but does not perform any analysis that we are including.

Dar et al.²⁷. The research problem is regarding passive enumeration and what type of data is enumerated and the value of it. This conducted research additionally determines which OSINT technique gathers the most data and information and is the most valuable. With regards to the information enumerated the information will be a similar type of information but refer only to DNS information and utilise a different method via Python Libraries to enumerate, our tool will also analyse the enumerated information to detect if a compromise has occurred and to identify subdomain takeovers.

Gong et al.²³ proposes a novel model which analyzes the reliability and validity of the OSINT data by using methods of comparative analysis. The research problem is of OSINT validity analysis and this literature provides a comparative analysis method uses Cyber Threat Intelligence (CTI) data and outputs. The collected CTI data results can be used as a basis for validation of various CTI data and their data feeds. This proposed model in this literature collected CTI data from CTI feeds through the use of network resources, we will enumerate DNS information automatically through a different means and perform analysis of the results which is where the gap in literature resides. This literature provided insights into CTI and OSINT information and collected data, there are no analysis or comparisons present in this literature.

Veciana et al.⁵ includes a research focus in this literature of demonstrating how Content Delivery Networks (CDNs) can be bypassed and evaded by accessing websites through their origin server directly. The research problem was identified for multiple ways CDN networks could be bypassed and in the research it also entails discussions of ways to aid in performing origin server discovery of misconfigured web servers hosting web applications and by leveraging several passive and active reconnaissance methods to do so. Our IR tool will also be coded with a widely adopted programming language except we will perform iteration to compare the DNS A record to the IP block range of the

company instead of just enumerating the valuable information.

2.10 INCIDENT RESPONSE

Incident Response is a key aspect of blue team defenders and enables defenders to stop attacks as they arise and protect systems in real time. This can often be done through setting up alerts on infrastructure to notify an incident responder whenever a threat is present. Our novel solution is in high regards to IR and we create one of these alert tools.

Wang et al.¹⁵. The research problem they address is concerning passive dns information being used for malware and IR capabilities which is a field lacking developments incorporating DNS information. The paper developed a security tool to collect local passive DNS data. The security tool developed is known as the 'PassiveDNS', a UI tool built for DNS enumeration which uses Splunk Enterprise and Amazon Web Services (AWS). Our research will utilise the same Cloud environment but we will not be using Splunk applications. We will be using DNS information enumerated however our DNS information will consist only of the DNS A record and the IP block range and provide IR detections based on a comparison.

Allen et al.²⁹ provides insights and novel research into the malicious network reconnaissance topic area for incident response. There is a research problem present within the industry with regards to NIDS security controls, they require rules and depending on the rules given, depends on the type of security threats picked up, this literature proposed a novel approach that produces a set of Network Intrusion Detection Systems (NIDS) detection rules which will identify any attempt of active reconnaissance on a network infrastructure so it can be detected automatically. Our solution only requires the DNS Answer record of a domain to perform the alerts and detections, which can be found through passive reconnaissance techniques, this literature focuses on active enumeration, opening a gap for passive enumeration techniques used to evade these kind of NIDS security rules.

Below is a comparison table 2.2 for all of the literatures reviewed for the topic of DNS enumeration, and our developed tool DNSDETECT. 'Tool Development' is a metric to assess if a tool was developed to infer our developments. 'Passive DNS Enumeration', check if passive recon is present, 'Performs Analysis' checks if the outputted data is used for analysis and 'Utilizing the Cloud' shows if the solution is deployed to a Cloud environment.

Publications	Tool Development	Passive DNS Enumeration	Performs Analysis	Utilizes the Cloud
Thorsell et al. ²⁶	x	✓	✓	x
Kothia et al. ¹	x	✓	✓	x
Baig et al. ¹⁴	x	✓	✓	x
Dar et al. ²⁷	x	✓	x	x
Wang et al. ¹⁵	✓	✓	x	✓
Carpene et al. ⁷	x	✓	x	x
Liu et al. ⁹	x	x	✓	x
Gong et al. ²³	x	✓	✓	x
Veciana et al. ⁵	✓	x	✓	x
Marques et al. ⁸	x	x	✓	x
Allen et al. ²⁹	x	✓	x	x
Our Work	✓	✓	✓	✓

Table 2.2: DNSDETECT Review Papers Table Comparison

Social engineering bypasses all technologies, including firewalls.

Kevin Mitnick

3

Architecture of the Security Tools

This section covers the design and architecture of the novel solutions InfoGath and DNSDETECT and identifies specific criteria and information relating to the development process of each novel solution, this section is regarding designing the solutions and outlining key information in relation to the development of these solutions.

3.1 INFOGATH

The first tooling system is relating to the Offensive Security domain and is specifically targeting public information gathering (OSINT) and is appropriately titled 'InfoGath' to reflect this activity. InfoGath will incorporate usage of a common certification third party, CRT.sh, a popular secure third party website hosting OSINT information on hundreds of domains including certificate information. The novel solution will return key valuable OSINT information that can be used within exploitation phases.

This system will combine various functionalities of OSINT techniques to return a collection of useful, important information sets displayed as 'collections' which can be used for mapping out digital perimeters of web application targets. It will automatically generate a graph for the data collected to represent the data. This information outputted will enable cyber security professionals to understand targets better and the data will provide insights into how the target interacts with other domains associated with it. This tool can help PTs in their security assessments against web applications through the outputted recon information this tool provides.

3.1.1 MOSCOW PRIORITISATION REQUIREMENTS

We implemented a Moscow Prioritisation technique for outlining the requirements of InfoGath. This is a popular method for identifying and listing the requirements of a project and is used within the industry for project development and design. We used this technique to clearly outline the requirements and provide insights into the design and architecture of the InfoGath novel solution. Moscow (MSCW) has four sectors of requirements split up per each non-vowel, as seen below.

M - Must Have:

- Python libraries (requests, bs4, socket, re, time, networkx and matplotlib.pyplot)

- Local Write Access
- Internet Connected Device
- Python 3.7.9 installed
- InfoGath will collect valuable, useful and correct OSINT information from in-scope targets
- Domain User Supplied Input.
- Enumerate at least 4 pieces of valuable OSINT information.
- The enumerated OSINT is stored to a CSV file.

S - Should Have:

- InfoGath will automatically generate a visual graph for the collected information
- The tool performs the OSINT automatically
- enumerate from a trusted third party OSINT domain

C - Could Have:

- The graph will connect all of the OSINT to the parent node
- The terminal output is clear, concise and understandable.
- Headings in the output prompts to differentiate the types of enumerated data

W - Will Not Have:

- An unlimited node graph creation due to library limitations and time frame
- Other relevant OSINT information enumerated, only actionable OSINT information that is solely valuable

Figure 3.1 shows a UML diagram connecting all of the MOSCOW requirements appropriately to form a complete UML diagram to identify how each section is connected together for the novel solution of InfoGath.

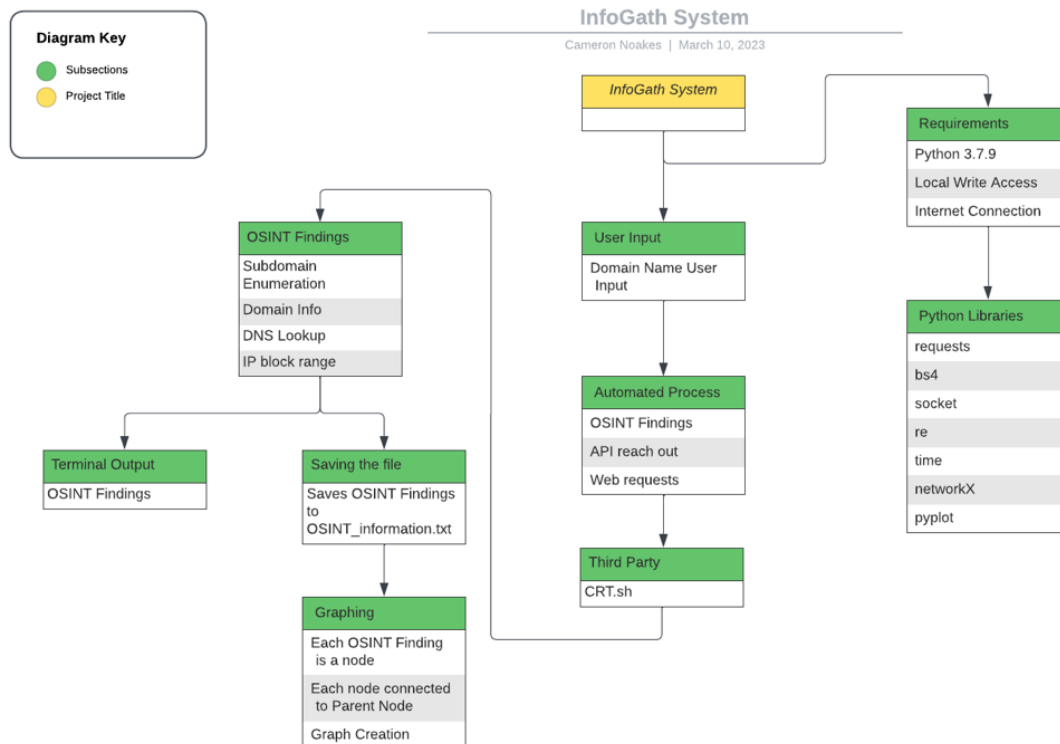


Figure 3.1: UML Diagram for InfoGath

3.2 DNSDETECT

The second tooling system is a Detection Engineering (DE) Incident Response (IR) tool titled 'DNSDETECT'. This tool provides a way to uniquely identify if a subdomain takeover is present, captures the present threat and relays it onto an incident responder to make aware, this tool runs every day as a cron job and was deployed to AWS cloud environment. DNSDETECT performs its duties by enumerating the DNS A record and generating the appropriate IP block range from it and compares the DNS A record to the IP block range to determine if it resides within the range.

DNSDETECT is the second novel solution development we undertook and will be an AWS cloud-based industry tooling system for Blue Team defenders to detect domain hijacking. If the

DNS Answer record is out of the IP address block range an alert of an indication of an attack present is sent. This unique way of detecting subdomain takeovers will provide a novel perspective and catch attacks uniquely. SNS and CloudWatch will be used to send detection response alerts frequently to an IR email account.

3.2.1 MOSCOW PRIORITISATION REQUIREMENTS

MOSCOW is used again for the requirements of DNSDETECT. We used this technique to clearly outline the requirements and provide insights into the design and architecture of the novel solution.

M - Must Have:

- Domain name variable. Used to identify the target domain.
- Python libraries (dnspython, dns, socket and boto3).
- Have the AWS account and environment correctly configured and an available internet connection
- Performs detections and responses when the DNS Answer record is outside of range
- Performs no detections when DNS Answer record is within the range
- DNSDETECT will correctly enumerate the DNS Answer record of any inputted domain
- The tool will generate the IP block range from the DNS Answer record through an iteration of 0-255.

S - Should Have:

- Correctly configured and deployed triggers of an SNS service and a CloudWatch Scheduler service.

- AWS Project running on Python V3.9
- DNSDETECT will compare the DNS Answer record to each instance in the IP block range via iterative linear search loop.
- The code should follow common coding practises

C - Could Have:

- the code should be commented
- The detection should run at least once a week
- additional information regarding the dynamic target within the email response
- Scalability, to be able to scale the project to multiple targets at the same time.

W - Will Not Have:

- Multiple targets per DNSDETECT project, one per project.

Below is figure 3.2 detailing the UML diagram for each of the requirements and features implemented and how they interact with one another and are linked, which collectively form the full novel solution of DNSDETECT.

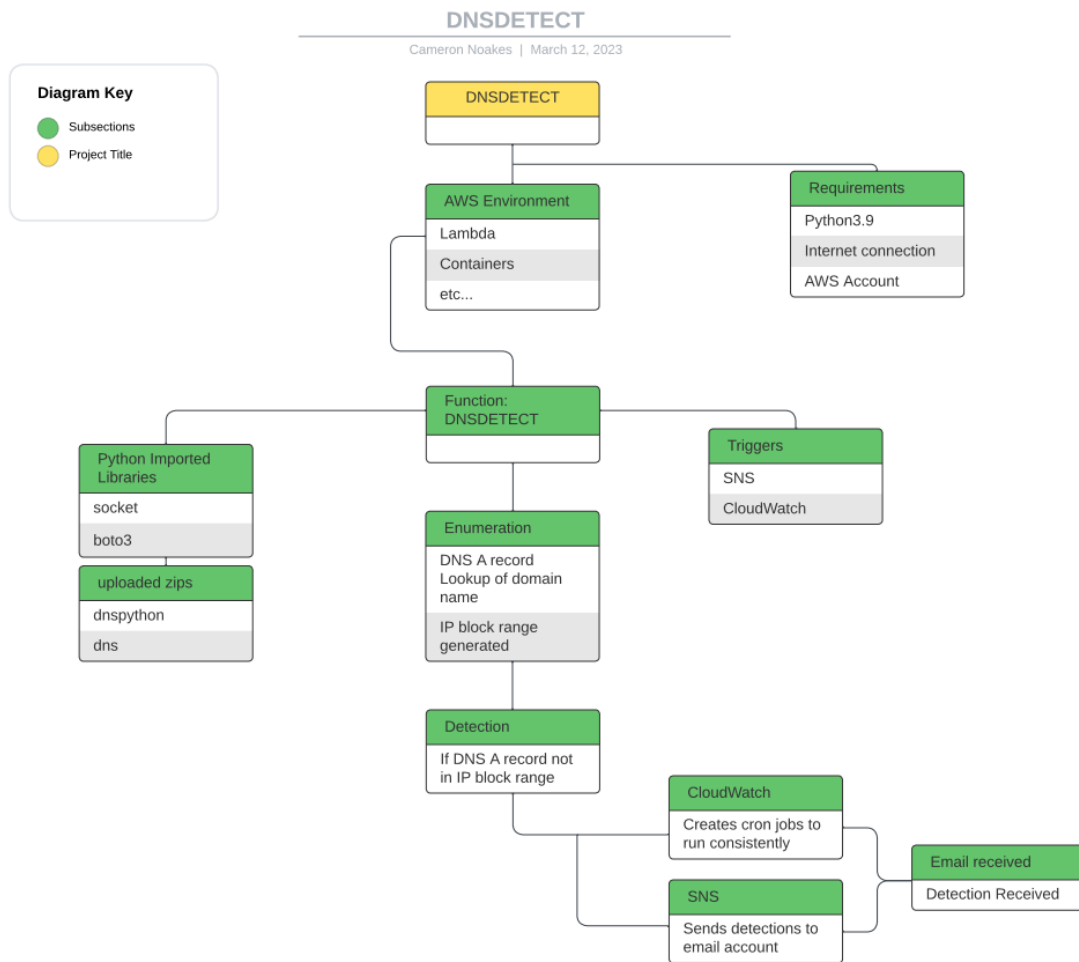


Figure 3.2: UML Diagram for DNSDETECT

3.2.2 A DOMAIN HIJACK CAN OCCUR VARIOUS WAYS DEPENDING ON THE SCENARIOS PRESENT. TWO COMMON METHODS ARE

- The attacker manages to compromise the IP address of the machine assigned the DNS A record, allowing the attacker to have control over the IP address of the DNS A record to issue malicious commands or actions (such as capturing confidential information from users visiting the site)

- The website certificates become expired or invalid on the targeted domain which enables a malicious threat actor to issue a self signed SSL certificate and take over the domain.

Overall, the attacker manages to compromise the website hosting, allowing the attacker to have control over hosting the domain, when the attacker re-hosts the domain it will be under one of their owned IP addresses, making the DNS A record not reside within the company range.

3.3 CVSS AND CVE THREAT MODEL

This section relates to DNSDETECT and is to be able to compare subdomain takeovers and domain hijacking vulnerabilities to an appropriately linked threat model that includes these types of attacks to provide further information about the attacks and how they occur and can be mitigated. These two areas of attacks are not specific vulnerabilities but rather an end result of combined vulnerabilities to eventually compromise an entire domain, it is a type of attack which can be done through multiple methods.

The chosen threat model is Common Vulnerabilities and Exposures (CVE), a threat model developed from the MITRE corporation and provides a database for every type of o-day identified with a unique CVE ID. A o-day is a new vulnerability in services or software that was previously unknown. The threat model details domain hijacking and subdomain takeover and returns 32 documented CVE records with attacks associated to this attack type. DNSDETECT does not utilise the CVE database but the attacks are represented within them. CVE also incorporates the Common Vulnerability Scoring System (CVSS) and provides severity ratings, remediation tactics and additional threat model intelligence on the outlined types of attacks.

Being hit by a cyber attack is going to be painful. But it can be less painful if you're prepared.

Tyler Hardison CTO, STN

4

Security Tool Developments

This chapter will cover everything relating to the code base developed, the methodologies used for the development, the code explained and the functionality of both of the novel security tools developed. This will be done for InfoGath first and then followed by DNSDETECT. The methodology was used to develop both tools and is discussed below.

The Methodology

The methodology used for this process was decomposition. Decomposition is the act of when undertaking large complex developments, to split up the entire project into manageable, sizeable chunks which the developer can focus on one at a time, it creates highly focused developments where each developer works on a small piece of the project. We used decomposition to enable this development and research process. Each section of the novel solution has been split up into various segments and each addressed as a separate isolated instance, enabling highly focused developments. It is the process of understanding the project well enough to delineate the smaller tasks to complete which, once put together can form the entire finished project and is the model we used to develop our novel solutions. The decomposition was used within both novel solution developments to split up the large projects based on functionalities of the tooling systems.

4.1 OFFENSIVE SECURITY TOOL 'INFOGATH' DEVELOPMENT

4.1.1 INFOGATH OVERVIEW

In this section we will be discussing the code base developed for the OSINT collector tool 'InfoGath'. This section will cover documenting the code, explanations, use cases with real life accurate data extraction and more. This section will provide a comprehensive in-depth understanding towards the code developed and will aim to alleviate any underlying questions surrounding the project and its functionality and usage.

4.1.2 FUNCTIONALITY OF INFOGATH

- Domain name registration information. Information relating to the domain registration such as who issues the domain, date of issue, company name and more are a part in the data extraction.

- Subdomain Enumeration: Passive third party API enumeration of subdomains is present and returned to the end user which is used to increase the attack surface.
- DNS Information Enumeration: DNS lookup requests are performed automatically on the given target domain to return valuable DNS reconnaissance (A, AAAA, NS, CNAME and MX records).
- IP Network Block Enumerator: Enumerates the entire IP block range of the company the target domain name belongs to, ideal for lateral movement and identification of other network targets.

From identification of the above functionalities of InfoGath, we can identify the multiple real world use cases a tool like this possesses and the level of security it can provide to medium and small scale organizations. This tool, although for Offsec, can also be utilised by any other security team such as Blue Team professionals to identify information leakage as OSINT.

Code Snippets and Explanations

There are 166 lines of code for the novel InfoGath tooling system. The code has been documented in this report as well as in-code commenting. For this report, the code explanations are given before the code is shown below. The in-line commented version of the code will be uploaded as a ZIP code file for submission.

Imported Libraries

We imported a series of Python external and built-in libraries to be able to produce the functionality intended which already had solutions incorporated to perform these requests, we utilise these libraries to conduct some of the OSINT. The imported libraries are listed below with each a brief description.

- **BeautifulSoup** - bs4 is used to web scrape information online and output from requests
- **requests** - This library enables HTTP web requests to be made, often used alongside bs4
- **networkX and matplotlib.pyplot** - A graph visual representation library that can be used for generating graphs and linkable diagram nodes automatically.
- **socket** - Socket API to send messages across a network and perform lookup requests
- **re** - Regular Expression library to match results to RegEx expressions
- **time** - enables delays in the terminal output

The below figure is of the InfoGath imports 4.1 from line 1 to 13 with a description for each import similar to the above list.


```

1  #Importing the necessary Libraries.
2  # 'requests' to make HTTP reqs, 're' for RegEx,
3  # 'socket' to make connections, pydot for analysis
4  # diagrams, 'bs4' for web scrapping and formatting
5  # and 'time' for delays to process information.
6  from bs4 import BeautifulSoup
7  import requests
8  import socket
9  import time
10 import re
11 import networkx as nx
12 import matplotlib.pyplot as plt
13

```

Figure 4.1: InfoGath Imported Libraries

get_subdomains

The function `get_subdomains` takes three input arguments the first being the domain name and then `check_active` and `output_file` which are set to inactive by default. The below function enumerates all of the related subdomains from CRT.sh API of the target domain, stores them within an array after a regex expression match to find them and strips HTML parse content out of the response to be left with the subdomains only. The subdomains are then printed out to the terminal one by one and subsequently stored to a file through the `write_to_file` function. This can be seen below in the `get_subdomains` figure 4.2

```

13
14 # Below function enumerates all subdomains from CRT.sh API of the target domain, stores them within
15 # an array after a regex expression match to find them and strips HTML parse content out of the response
16 # to be left with the subdomains only. The subdomains are then printed out to the terminal one by one
17 # and subsequently stored to a file through the write_to_file function.
18 def get_subdomains(domain, check_active=False, output_file=None):
19     url = f"https://crt.sh?q={domain}"
20     r = requests.get(url)
21     subdomains = []
22     regex = r"^([a-zA-Z]{1})|([a-zA-Z]{1}[a-zA-Z]{1})|([a-zA-Z]{1}[0-9]{1})|([0-9]{1}[a-zA-Z]{1})|([a-zA-Z0-9]{1})|([a-zA-Z0-9]{1}[a-zA-Z]{1})|([a-zA-Z0-9]{1}[0-9]{1})|([0-9]{1}[0-9]{1})"
23     jump = r.text.replace("<BR>", "\n")
24     nohtml = re.sub("<.*?>", "", jump)
25     nohtml = nohtml.replace(" ", "")
26     matches = re.finditer(regex, nohtml, re.MULTILINE)
27     for match in matches:
28         subdomains.append(match.group())
29     subdomains = list(set(subdomains[1:]))
30     time.sleep(0.5)
31     print(f"Subdomains found with crt.sh:")
32     time.sleep(1)
33     for sub in subdomains:
34         print(f"[+] {sub}")
35         write_to_file(sub)
36         time.sleep(0.07)
37
38

```

Figure 4.2: Get subdomains Function With Description

get_domain_info

This function obtains domain certificate information on the inputted target domain, specifically, the issuer name (third party registration) and common names relating to the subdomain and performs a GET request for this via the CRT.sh API. This outputs the CRT columns of 6,7 values that being 'Issuer Name' and 'Common Name' which are then outputted and written to the file. The figure below is of the domain information enumeration function 4.3

```

38
39 #####
40 # Function: Obtains domain certificate information on the target,
41 # specifically, Issuer name (third party) and common names as a POC
42 # to identify that the correct domain was chosen.
43
44 # Performs a 'get' http req for CRT.sh and query parameter and appends
45 # inputted domain onto the end and outputs column 6,7 values and returned.
46 # 'Issuer Name' and 'Common Name' are these columns and are also written
47 # to the same file.
48 #####
49
50 def get_domain_info(domain):
51
52     response = requests.get(f"https://crt.sh/?q=%.{domain}")
53     soup = BeautifulSoup(response.text, 'html.parser')
54     rows = soup.find_all('tr')
55
56     for row in rows:
57         cols = row.find_all('td')
58         if len(cols) >= 6:
59             common_name = cols[6].text
60             issuer = cols[7].text.replace(", ", "") #.replace removes the default
61             #commas which would cause issues with the visual representation graph.
62             time.sleep(1)
63             write_to_file(common_name)
64             write_to_file(issuer)
65             print ("")
66             print ("DOMAIN CERTIFICATE INFORMATION - Issuer Name and Common Name:")
67             print (f"Common Name: {common_name}\nIssuer: {issuer}")
68             return

```

Figure 4.3: Get Domain Info Function With Description

dns_lookup

This function performs a DNS lookup request for five records associated with DNS and the results are stored in an array and utilises the socket library to perform these DNS lookup resolve requests. The information is then outputted and written to the file. The figure referenced below is the DNS lookup request function 4.4

```

69
70 #####
71
72 # This function performs a DNS lookup request for 5 records associated with DNS
73 # the results are stored in a {} array and utilises the socket library to perform
74 # these lookup requests. Then the information is outputted and written to the file.
75 def dns_lookup(domain_name):
76
77     result = {}
78     result['A'] = socket.gethostbyname(domain_name)
79     result['AAAA'] = socket.gethostbyname_ex(domain_name)[-1]
80     result['CNAME'] = socket.gethostbyname_ex(domain_name)[0]
81     result['NS'] = socket.gethostbyname_ex(domain_name)[1]
82     result['MX'] = socket.gethostbyname_ex(domain_name)[2]
83     print ("")
84     time.sleep(1)
85     write_to_file(result)
86     print ("DNS INFORMATION (A, AAAA, CNAME, NS and MX Records):")
87     print ("Domain Name:", domain, "results of:", result)
88
89

```

Figure 4.4: DNS Lookup Request Function With Description

get_ip_block_range

The below function enumerates the IP block range of the target domain inputted utilising the socket library and strips the last octet and combines it with a '.o/24' notation to form a subnet in CIDR notation to the end. This is to identify the IP block range which is then outputted and also written to the file. /24 is common for subnets and since our tool is used for use cases resolving to small and medium sized companies, this is ideal and accurate. The IP block range function can be seen below as a figure of the IP block range enumerator 4.5

```

90 #####
91
92 # The below function enumerates the IP block range of the target domain
93 # inputted utilising the socket library and to split the last octet and
94 # append a subnet of /24 to the end to demonstrate the IP block range
95 # which is then outputted and also written to the file.
96
97 def get_ip_block_range(domain):
98     try:
99         ip_address = socket.gethostbyname(domain)
100         ip_block = ip_address.split(".")[0:3]
101         print ("")
102         time.sleep(1)
103         print ("IP BLOCK RANGE OF TARGET:")
104         subnet = ".".join(ip_block) + ".0/24"
105         print (subnet)
106         write_to_file(subnet)
107
108     except socket.gaierror:
109         return None

```

Figure 4.5: Enumerate IP Block Range Function With Description

write_to_file

This code is the function that writes the enumerated data to a file in local storage named OS-INT_information.txt. Each element added having a commas after it to separate each added value, ready for the node-link graph representation. Any data is converted to string-type before writing to the file. This function is called after every function that provides functionality to ensure all enumerated OSINT information is appropriately written to the file with reference to the write to file function figure 4.6 seen below.

```

110
111
112 # Writing to the text file. Stores all enumerated OSINT information
113 # with each element added having a commas after it to seperate ready
114 # for the node link graph representation
115 def write_to_file(data):
116     with open("OSINT_information.txt", "a") as file:
117         file.write(f"{data},")
118

```

Figure 4.6: Write to File Function With Description

create_jpg_diagram

This function, when called, generates automatically the graph visual representation with the center node in the middle connected to all other nodes of OSINT information enumerated. This produces an informative graph. It reads the written file and extracts the content then sets them as nodes and adds edges. This is all in reference to the code seen as the figure below of the graphing function 4.7

```

119
120 #####
121 #ANALYSIS - Graphing:
122
123 # This function generates automatically the node-link graph visual
124 # representation of the given domain name target and all of the
125 # information enumerated to produce a well rounded informative
126 # graph to showcase to non-technical professionals. It does so by
127 # reading the written to file and extracting line by line the
128 # elements and sets them as a node and performs a linkage from each
129 # node to the parent centre node and adds edges as the links and saves
130 # locally and sets a default figure size of 10x10.
131
132 def create_jpg_diagram(file_name, parent):
133     G = nx.Graph()
134
135     with open(file_name, 'r') as file:
136         for line in file:
137             elements = line.strip().split(',')
138             for element in elements:
139                 G.add_node(element)
140                 if element != parent:
141                     G.add_edge(parent, element)
142
143     plt.figure(figsize=(10,10))
144     nx.draw(G, with_labels=True)
145     plt.savefig('OSINT_diagram.jpg')
146
147     print ("")
148     print ("VISUAL REPRESENTATION:")
149     print ("Analysis Image Created, see current directory!")
150

```

Figure 4.7: Graph Generator Function With Description

The Variables and Callouts

A domain input is present for the end user to enter the target domain to perform OSINT enumeration on as well as function callouts for each separate function which provides functionality with the domain entered parsed as an argument. Below the function callouts is some initialisation variables present to provide a name for the written to file, an image name initialised for the graph generation and a final function callout to the graph generation function. Below a figure is present detailing the initialisation variables and function callouts 4.8

```

151 # User input for the target domain to OSINT scrape and enumerate.
152 domain = input("Enter a domain to start the process (please exclude prefixes i.e 'www.'): ")
153
154 #Function callouts
155 get_subdomains(domain)
156 get_domain_info(domain)
157 dns_lookup(domain)
158 get_ip_block_range(domain)
159
160 # Pre-set values.
161 file_path = 'OSINT_information.txt'
162 image_path = 'OSINT_Data_diagram.jpg'
163 create_jpg_diagram("OSINT_information.txt", domain)
164
165
166 # Cameron Noakes

```

Figure 4.8: Variables and Callouts to the Functions

The Text File

The text file is the file all of the enumerated OSINT information is written to and is used with the graph generation for the visual representation part of the novel solution. A snippet of the text file contents on an example domain can be seen below along with the auto-generated graph system. Below are two figures, one detailing the text file contents 4.9 and the other detailing the graphing visualisation generated 4.10

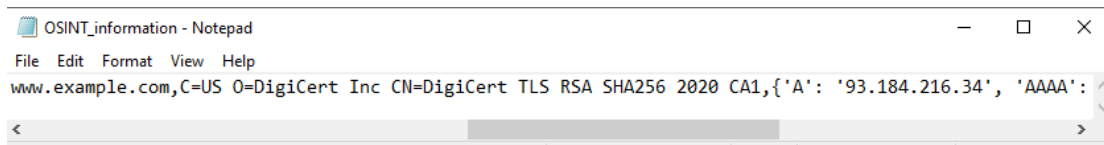


Figure 4.9: Write to File Contents With Example Data

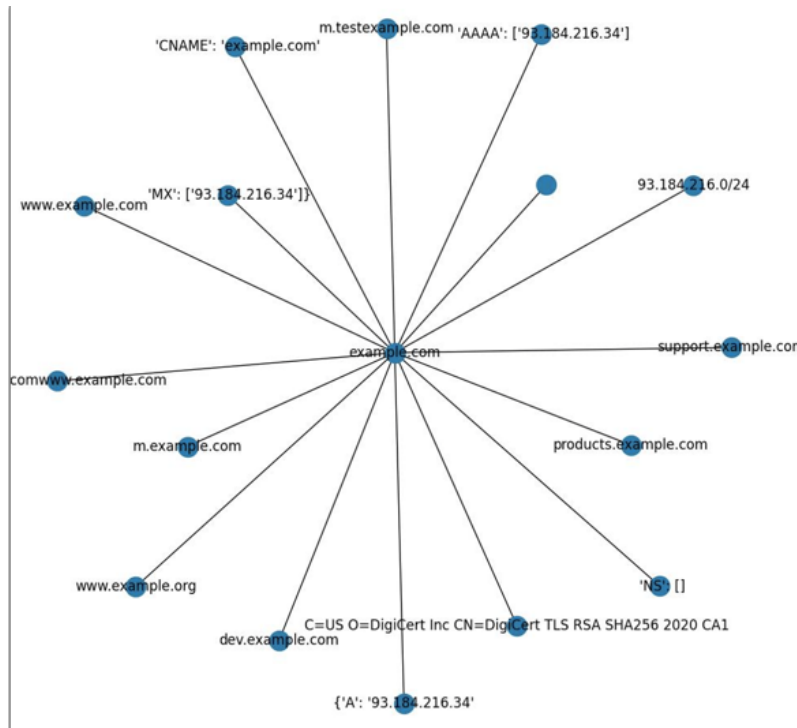


Figure 4.10: Graphing of example.com

4.1.3 CHALLENGES FOR INFOGATH

- PyDot. The initial development of the program used the PyDot library to generate the visual representation but a bug resided within the library which was identified after development, the bug did not allow the text file and the graph visualisation to be created within the same program due to the library mistaking the text file to still be open which is when we switched to use NetworkX and PyPlot instead.

- HTML Parsing. The CRT.sh API requests contained HTML code, specifically tags, which would have needed to be stripped, this posed a challenge but once the regex expression was correctly developed worked as expected and stripped off HTML tags.
- Minor adjustments. There were several minor adjustments I had to include in the code to be able to function as expected, one of the minor examples were to strip the 'issuer' output of cert information of commas to have the graph generate properly.

4.1.4 LIMITATIONS FOR INFOGATH

- The visualisation libraries chosen (NetworkX and PyPlot) have a maximum node limit, the maximum node count is large, but not infinite, only allowing a maximum of medium scale organisational domain targets.
- Third Party availability. Functionality utilising the CRT.sh API relies on the availability of CRT website to perform these requests, CRT is a valid, trusted third party for certificate and domain information storage which is unlikely to drop.

4.1.5 WAS INFOGATH SUCCESSFUL?

The InfoGath automated security tool accurately takes a singular user input of a domain name and processes the input and completes various functionalities based on this input to enumerate OSINT information and generate visual representations of the data. This tool provides real world insights and impact into the art of Offensive Security and can utilise this tool to provide them with a collective use of important OSINT information.

4.1.6 FUTURE WORK

The future work for InfoGath would be to incorporate more third parties for enumeration and recon activities to enable a diverse perspective, third party APIs could also be utilised such as Wapalyzer (A popular tool to detect front and back-end technologies on web applications). Our novel solution of InfoGath could in the future additionally incorporate active enumeration methods to have more use cases and provide more data insights and OSINT enumeration.

InfoGath could additionally be deployed to a server within a company's infrastructure and provide ssh access into the server to run the program from a central dedicated environment instead of locally, enabling all security professionals an environment dedicated to this activity.

4.2 INCIDENT RESPONSE TOOL 'DNSDETECT' DEVELOPMENT

4.2.1 DNSDETECT OVERVIEW

The defensive security tool DNSDETECT, will be deployed in an Amazon Web Services (AWS) Cloud environment to detect subdomain takeovers and domain hijacking and communicate incident detection alerts an Incident Response email account belonging to blue team security. DNSDETECT is a novel detection solution that has a variety of features used within the AWS Lambda Cloud environment. This tool can be used consistently to be able to quickly identify any domain compromises from a unique novel perspective that has not been seen before. DNSDETECT was developed using AWS Lambda and has two triggers attached to mark as Cron Job (frequent running processes) and integrated with email communication functionality. DNSDETECT uses AWS Lambda, has three functions and two triggers of Simple Notification Service (SNS) and Cloud-Watch with library usage of boto3 and socket.

4.2.2 FUNCTIONALITY OF THE TOOL

The functionality of DNSDETECT is to be able to provide Incident Response for web application compromises, specifically for domain hijacking. The tool has a hard coded domain name variable which will act as an input of the target domain and enumerate the domain name DNS Answer (A) record through a DNS lookup and then generate the IP block range from the Answer record. A comparison of each element in the IP block range is present to determine if the DNS Answer record is within the IP block range. If the DNS Answer record is outside of the company's range, this could indicate a compromise and will alert a dedicated SMTP email account depending on when the Cron job is set to run. This creates various use cases to protect domains from a category of various attacks.

4.2.3 AWS ENVIRONMENT CONFIGURATION

At the start of the development process, we setup a new Amazon Web Services (AWS) account to enable us access to AWS Lambda, an IDE-like environment to deploy code and include various Cloud-based features. AWS Lambda allows developers to run arbitrary code within a dedicated cloud infrastructure. Inside AWS Lambda, we setup a new function (project template) named DNSDETECT for our development environment and set the server to eu-west-2 (London).

AWS Lambda contains a default `lambda_function.py` template (renamed to `mycode.py`) with a function handler to link to the AWS environment, taking no arguments. This provided a clear template to develop DNSDETECT.

4.2.4 THE CODE

Below is the decomposed code explained in full detail to allow understanding of the functionality and how the code works. It is important to understand the code before we utilise the Cloud environment for its full capabilities.

get_ips_from_dns

We imported two commonly used libraries of boto3 for connecting DNSDETECT to the SNS service provided by AWS and socket to perform the DNS Answer record lookup request

get_ips_from_dns is a function to execute a DNS lookup request using the socket imported library to be able to identify the DNS Answer record of the inputted target domain. The function takes the domain name as an argument and extracts the appropriate record and saves to the ips variable. The figure below is of the get_ip_from_dns function 4.11.

```

1 import socket
2 import boto3
3
4 # Enumerating IPv4 address of domain
5 def get_ips_from_dns(domain_name):
6     try:
7         # Use the socket library to perform a DNS lookup on the domain_name
8         answers = socket.getaddrinfo(domain_name, None)
9
10        # Extract the IPv4 addresses from the answers
11        ips = [answer[4][0] for answer in answers if answer[0] == socket.AF_INET]
12
13        # Return the unique IPv4 addresses from ips variable
14        return list(set(ips))
15
16    # Exception handling of errors.
17    except Exception as e:
18        print("Error performing DNS lookup:", e)
19        return []

```

Figure 4.11: get IP from DNS

generate_ip_block

This function takes the ips variable which was previously re-defined which stores the DNS Answer record. ip_block variable stores all the IP addresses within the range associated with the DNS Answer record. The last octet of the DNS Answer record is stripped and replaced by an iterative loop to generate all hosts within the IP block range. The figure below shows the IP block generation function

4.12

```

20
21 # Enumerating the IP block range
22 def generate_ip_block(ips):
23     ip_block = []
24     # Splits the IP per '.' and generates all 0-255 host outcomes.
25     for ip in ips:
26         ip_parts = ip.split(".")
27         for i in range(1, 256):
28             new_ip = ".".join(ip_parts[:-1] + [str(i)])
29             ip_block.append(new_ip)
30     return ip_block
31

```

Figure 4.12: IP Block Generation Function

lambda_handler

This is the default function that configures the triggers and performs the comparison against the IP block range, this is done by comparing the DNS Answer record to each element in the generated block range. Below is the figure referencing the Lambda function 4.13.


```

32
33 def lambda_handler(event, context):
34
35     # Enter the domain name
36     domain_name = "example.com"
37
38     # Connecting to the SNS service created.
39     client = boto3.client(
40         "sns",
41         aws_access_key_id = "[REDACTED]",
42         aws_secret_access_key = "[REDACTED]",
43         region_name = "us-east-2"
44     )
45
46     # Calls the intended functions.
47     ips = get_ips_from_dns(domain_name)
48     ip_block = generate_ip_block(ips)
49
50     # Test Objectives ###
51     # --- remove comment below to make 'A' record outside range ---
52     #ips = ['87.34.24.112']
53
54     # Checks if ips (A record) is in IP block range,
55     # if so, compromise detected, alert recieved.
56     found = False
57     for ip in ips:
58         if ip in ip_block:
59             found = True
60             break
61     break
62
63     if not found:
64         client.publish(
65             TopicArn = "arn:[REDACTED]:dns_tooling",
66             Message = "#####\n" + domain_name + "\n#####\n\n"
67         )

```

Figure 4.13: Lambda Function

4.2.5 AWS TRIGGERS

We used Simple Notification Service (SNS) and CloudWatch to provide a system for detecting threats in a novel way. SNS enables email connections and send emails to a dedicated email account for Incident Response to inform professionals of a potential compromise. Below is the figure detailing the attached triggers 4.14

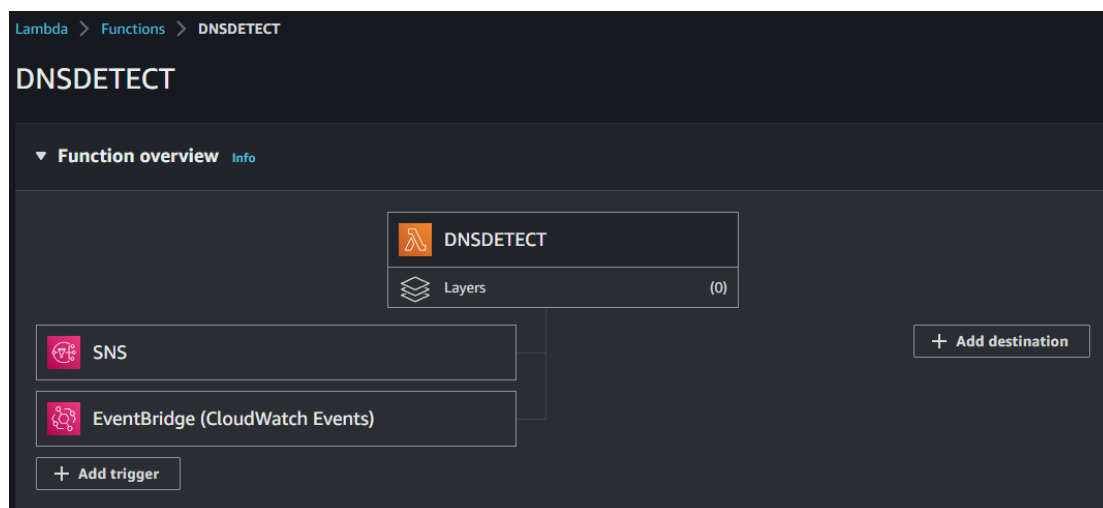


Figure 4.14: The Triggers Attached to DNSDETECT

4.2.6 IoC DETECTION EMAIL

A brief description for the alert, DNS A record and IP block range are present in the detection email. Our tool does not suffer from any false positives. Below is a figure detailing the detection email received 4.15

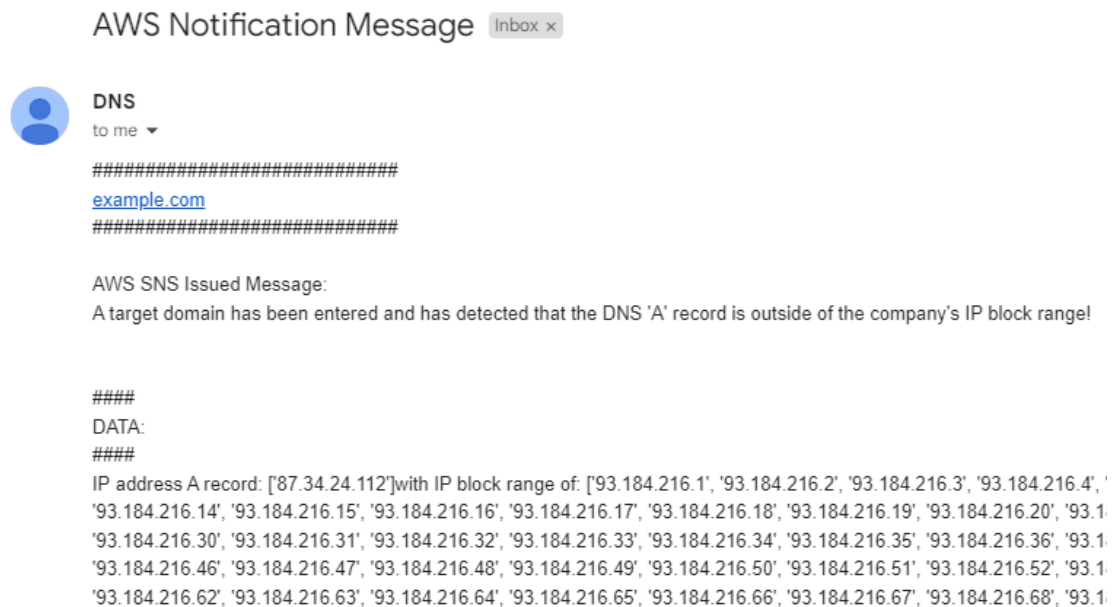


Figure 4.15: The Detection Email Response

STATEMENT

DNSDETECT has now been discussed fully at length with the steps explained and the functionality tested, this cloud project is novel and provides a detection and response system for multiple targets at any one time and keeps the Incident Response team up to date with new IoCs. This can detect real life use case threats when they arise regardless of geographical location due to being deployed in the cloud. This AWS project did not come without challenges and limitations.

4.2.7 CHALLENGES FOR DNSDETECT

- Generating IP block range from Answer record. This was the main challenge due to the need for an iterative process based on a previous input result, after this challenge was overcome it worked as expected.

- Detection Emails. We would receive the correct result of a detection email when the DNS Answer record is outside of the IP block range, however a logic error prevented the emails from stopping, producing a continuous stream of 500+ emails since with AWS Lambda, once ran you cannot stop the process.
- Overwriting IPS. The variable 'ips' holds the value of the DNS Answer record, this is then subsequently compared to the IP block range of the domain name. To generate a test event, we had to change part of the program to make the DNS Answer record out of the range as seen on line 52. Originally would notify us if within the range, which had the opposite desired affect, we adapted the comparison to alert us if and only the Answer record was outside of the range.

4.2.8 LIMITATIONS FOR DNSDETECT

- Access to AWS and Email account. For this Incident Response project there needs to be an email account for receiving the Incident Response alerts, a relatively easy requirement to satisfy as well as access to the AWS Cloud environment
- Library Modules. DNSDETECT relies on four library modules for Python (boto3, socket, dns, dnspython). If the modules were deprecated in the future this could identify an issue with the correct functionality of the project, which would need the new version import instead. A simple fix but still a limitation.

4.2.9 WAS DNSDETECT SUCCESSFUL?

DNSDETECT is a dedicated security tool for Incident Responders to identify IoCs within a reasonable time frame and can act accordingly to protect infrastructure and systems. This project was deployed to AWS, enabling the functionality of cloud environments to be incorporated into the

development of DNSDETECT. DNSDETECT can be used within the industry to protect web application infrastructure of company domains by a novel attempt at cyber protection that no other tool has discovered before.

4.2.10 EVALUATION FOR DNSDETECT

Below is the requirements set out in the design chapter and if these requirements were met to determine an accurate evaluation of our novel solutions by comparing to the requirements we set out before development. All of the requirements outlined in the design section were implemented and worked as expected, we managed to get a tool with full requirements met.

- Python libraries. The Python libraries used were the same as outlined in the design section and no additional ones were needed to be included.
- AWS environment configured. We have the cloud environment configured as intended, using the same triggers outlined in the design chapter.
- DNSDETECT has accurate detection and response capabilities and intended functionality. This requirement was fully met.
- IP block range iteration. We were successfully able to complete this requirement by iterating through a loop of 0-255 to generate the entire IP block range.

4.2.11 USE CASES AND EXAMPLES

There are multiple use case examples for DNSDETECT especially for Incident Response since it is widespread and pervasive. A few examples of potential main use cases are given below but not restricted or limited to these identifying examples.

- In-house IR. DNSDETECT can be used within a company to help secure their external facing web application assets which are often open to the internet.
- Cyber Security Consultancies. Can be used to protect clients from web application cyber threats on their web infrastructure and provide a positive reputation for the client.
- Freelance. If any individual with a website (business or otherwise) wants to keep up to date with if their website has been compromised, they can utilise DNSDETECT.

4.2.12 FUTURE WORK

The future work of DNSDETECT is to have future implementation of additional features to fully utilise performing these critical compromise detections. This security tool could be built out to directly communicate with bespoke Incident Response systems for in-house use. This future work would provide further positive impact to the industry and in the act of securing valuable digital assets.

Teams that say their cyber-security is really good are the ones to worry about.

Dame Dido Harding, former CEO of TalkTalk

5

Experimentation

In this section, we will compare and discuss our novel solutions and similar tools, the tools will be compared based on various factors and metrics. This section will compare our developed tools to that of industry standard tools already existing to identify how our developments are novel and add value.

5.1 SUBDOMAIN ENUMERATION - SUBLIST3R (FOR INFOGATH)

Relating to our novel solution InfoGath, we can compare the metrics alongside a popular subdomain enumeration tool known as Sublist3r. Sublist3r uses various search engines (Google, Bing, Ask) to enumerate subdomains and also utilises third parties such as Netcraft and Virustotal. We conducted an experiment on example.com to compare output enumeration metrics. What InfoGath provides that Sublist3r doesn't, is a dedicated singular third party to enumerate from. InfoGath also provides multiple functionalities outside of just subdomain enumeration, enabling a higher use case.

Below is an example of the functionality comparison between Sublist3r and InfoGath, labelled figure 5.1 and figure 5.2.


```
(root@kali)~/home/kali/Sublist3r
# python3 sublist3r.py -d hackerone.com

Sublist3r

# Coded By Ahmed Aboul-Ela - @aboul3la

[-] Enumerating subdomains now for hackerone.com
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[!] Error: Virustotal probably now is blocking our requests
[-] Total Unique Subdomains Found: 4
www.hackerone.com
api.hackerone.com
docs.hackerone.com
support.hackerone.com

(root@kali)~/home/kali/Sublist3r
```

Figure 5.1: Sublist3r Scan on Hackerone.com

```
>>> %Run InfoGath_Submission.py

Enter a domain : hackerone.com
Subdomains found with crt.sh:
[+] mta-sts.hackerone.com
[+] ssl333694.cloudflaressl.com
[+] hackerone.com
[+] enorekcah.com
[+] ssl581118.cloudflaressl.com
[+] gslink.hackerone.com
[+] ssl383281.cloudflaressl.com
[+] ssl383282.cloudflaressl.com
```

Figure 5.2: InfoGath Scan on Hackerone.com

As seen above in the figure 5.1, Sublist3r correctly enumerated a total of four subdomains relating to the target domain Hackerone.com across eleven (11) third parties and tools. Referencing figure 5.2, we can see with our one third party of CRT.sh, we have enumerated many more subdomains. Some of these domains would be false positives but with the enumeration of as many subdomains as possible, there is a higher chance of finding a hidden subdomain that other tools like Sublist3r overlooked, providing an additional attack vector. There are various differences between Sublist3r and the novel solution InfoGath, some being that Sublist3r takes a while to conduct the subdomain searches and some of the relied on services error out as seen in the Sublist3r figure above

with VirusTotal. Sublist3r also checks five search engines when they most likely have the same index information, causing exhaustion of resources and duplicate searching taking more time. Sublist3r also requires prior knowledge of command line and arguments to run whereas InfoGath doesn't and also provides visual data representations. These are just some of the many differences and novel areas we developed to provide a more well rounded multi-functional novel solution.

5.2 INFORMATION ENUMERATION AND VISUALISATION - MALTEGO (FOR INFOGATH)

Maltego is a graphical User Interface (UI) security tool that provides insights into data, performs collections of data and provides visual graph representation of the data. Maltego maps the information enumerated into a graph to become easier to visualise and identify how each node is linked. Maltego requires a license to use commercially, restricting the evaluation process and due to open-source evaluations, InfoGath provides open-access enabling more users to utilise InfoGath.

5.3 INFOGATH TOOL COMPARISON

Summaries of the two best industry tools relating to InfoGath and its functionalities have been discussed above, we can compare their utilities, functionalities and metrics in the below table 5.1.

Tool	Multiple Functionalities	No False Positives	Other OSINT	Passive Enumeration
Sublist3r	x	x	x	x
Maltego	✓	✓	✓	x
Our Work	✓	✓	✓	✓

Table 5.1: InfoGath Tool Comparisons

5.4 DNS ENUMERATION - DNSRECON (FOR DNSDETECT)

DNSRecon is a powerful tool for DNS information enumeration and is used within the industry to conduct security assessments such as Penetration Testings, this is due to a security professional obtaining as much information at their disposal as possible, for DNSDETECT this is not necessary since the tooling system only requires a parent domain name to conduct all of its automated assessments. We can analyse the effects of DNSRecon and DNSDETECT on enumerating the DNS A record as seen below in figures 5.3 and 5.4.

```

[*]      TXT example.com v=spf1 -all
[*]      TXT example.com wgyf8z8cgvm2qmxpnbldrcltvk4xqfn
[*] Enumerating SRV Records
[+] 0 Records Found

(root@kali) - [/home/kali/Sublist3r]
# dnsrecon --type std -d example.com

```

Figure 5.3: DNSRecon example.com Answer record

```

example.com
#####

AWS SNS Issued Message:
A target domain has been entered and has detected that t

####
DATA:
####
IP address A record: ['87.34.24.112']with IP block range of

```

Figure 5.4: DNSDETECT example.com Answer record

Above shows both figures detailing DNSRecon and DNSDETECT, DNSRecon attempts to perform std record enumeration (which covers all DNS record lookup queries) but fails, it does sometimes provide a DNS Answer record but has a higher delay time and depends on the tools backend. DNSDETECT performs not only the DNS Answer record correctly, but efficiently and also provides a comparison with the IP block range of the company the DNS Answer record belongs to. DNSDETECT is a novel Incident Response system, but to compare to DNSRecon, an enumeration tool, we only evaluated DNSDETECT on the partial functionality to enumerate the DNS Answer record to compare effectively and fairly. DNSDETECT outperformed the enumerated record query within time frame, efficiency and accuracy and utilised the socket library in Python to accomplish this. Be aware that the email response we received from DNSDETECT has an incorrect DNS Answer record due to it having to be outside the range to receive the alert, this is for test

purposes of a hijacking attempt and was previously discussed in Chapter 4.

5.5 DNS ENUMERATION - DIG (FOR DNSDETECT)

Dig is a command line Linux terminal command to query DNS information of targets inputted. You can install Dig on Windows however you first need to adapt the installation setup and install BIND, this can become cumbersome and depends on the Operating System the end user is on. Within DNSDETECT, it is easier to utilise since the DNS lookup of the Answer record uses the socket library which is a pre-installed python library that only requires one line of code to import and have working.

We compared the same search between dig and the socket lookup request library, dig produced a very fast response and provided additional information, not just the DNS Answer record. DNS-DETECT performs the same request utilising the socket library and takes 794.89 ms whereas dig took 59ms. Dig is more efficient and faster but provides other non-relevant information to the direct record query, since we only required the DNS Answer record, although the socket library took longer, it provided the DNS Answer record only, removing issues with how the code should identify the DNS Answer record. These requests from dig and socket (within DNSDETECT) can be seen in the figures 5.5 and 5.6 below.

```

(root@kali)~/home/kali/Sublist3r
# dig example.com -t A

; <<>> DiG 9.17.19-3-Debian <<>> example.com -t A
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 60950
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                IN      A
;; ANSWER SECTION:
example.com.                 80615   IN      A      93.184.216.34
;; Query time: 59 msec
;; SERVER: 194.168.4.100#53(194.168.4.100) (UDP)

```

Figure 5.5: Dig example.com Answer record

```

Version: $LATEST
Duration: 794.89 ms

```

Figure 5.6: DNSDETECT example.com Answer record

5.6 DNSDETECT TOOL COMPARISON

We have chosen two popular DNS lookup and recon industry tools to compare to our novel solution of DNSDETECT, DNSDETECT is bespoke and unique in the way it identifies Incident Response detections that other industry tools don't incorporate (providing its novelty). Incident Response systems often are bespoke and closed source, thus instead of providing IR comparisons we provided similar functionality comparisons. These can be assessed below in the table 5.2.

Tool	Cloud-Based	Provides Detections	DNS Enumeration	Performs Lookup Requests
DNSRecon	x	x	✓	✓
Dig	x	x	x	✓
Our Work	✓	✓	✓	✓

Table 5.2: DNSDETECT Tool Comparisons

5.7 EXPERIMENT CONCLUSION

Both InfoGath and DNSDETECT novel solutions provide unique perspectives on multiple sets of enumerated data and perform functionalities that other industry tools have absent, this enables more enumeration of OSINT information and (for DNSDETECT) unique detections for identifying Indications of Compromise. These tools can be deployed within the industry to help gather OSINT information on web application targets (for InfoGath) or provide valuable detections for organizations to determine web application compromises with DNSDETECT.

*"Data is the pollution problem of the information age,
and protecting privacy is the environmental challenge."*

Bruce Schneier

6

Conclusion

To conclude on this research, we appropriately developed two novel solutions, one for Offensive Security within the topic area of OSINT enumeration and another for Incident Response teams within the topic area of domain hijacking and preventing it. These solutions filled an open gap within research and literature identified from chapter two. This gap was closed through the developments of InfoGath and DNSDETECT. These tools met all of the outlined requirements stated in chapter three which the development was documented in chapter four.

InfoGath provided valuable OSINT enumeration that can be utilised to view the attack surface better and use the information enumerated to help exploit web applications more effectively with the online information found.

DNSDETECT provided a novel way to identify domain hijacking and takeovers and utilised a cloud-based environment with email detection responses, this tool can be used widely across the industry and be deployed at scale for unique, novel perspectives to catch threat actors hijacking web applications.

The problem outlined for InfoGath solution was that there was no tool out there to obtain these valuable OSINT collections from one standalone tool, most of the tools are specific to one purpose and not broad, returning as much valuable OSINT as possible to aid in the knowledge of the target. Some tools would only provide subdomain enumeration, making the security professional have to utilise multiple tools to find the information of relevance.

The problem outlined for DNSDETECT solution was that there is no DNS publicly available tools within the industry to identify subdomain takeovers and hijacking through the use of analysing the DNS Answer record, this gap was identified for the industry and the gap filled with the DNSDETECT solution which identified a unique perspective on finding IoCs.

This research posed crucial and novel to the evaluation of pre-existing industry tools to be able to then develop our own which close industry gaps. This research identified various industry gaps within various topic area themes which all were included within both of our novel solution developments. Two novel solutions being developed was more than required for this dissertation, however, we acknowledged two major gaps within the industry and instead of focusing on one solution, we built two since we could not pass up an opportunity to provide double the value to the industry.



Appendix

A.1 APPENDIX A

This appendix A details the full code program of the InfoGath novel solution which can be seen below.

```
#Importing the necessary Libraries.  
# 'requests' to make HTTP reqs , 're' for RegEx,
```

```

# 'socket' to make connections , pydot for analysis
# diagrams , 'bs4' for web scraping and formatting
# and 'time' for delays to process information .
from bs4 import BeautifulSoup
import requests
import socket
import time
import re
import networkx as nx
import matplotlib.pyplot as plt

# Below function enumerates all subdomains from CRT.sh API of
# the target domain , stores them within
# an array after a regex expression match to find them and
# strips HTML parse content out of the response
# to be left with the subdomains only . The subdomains are then
# printed out to the terminal one by one
# and subsequently stored to a file through the write_to_file
# function .
def get_subdomains(domain , check_active=False , output_file=None
):
    url = f"https://crt.sh/?q={domain}"
    r = requests.get(url)
    subdomains = []

```

```

regex = r"^([a-zA-Z]{1})|([a-zA-Z]{1}[a-zA-Z]{1})|([a-zA-Z]{1}[0-9]{1})|([0-9]{1}[a-zA-Z]{1})|([a-zA-Z0-9] [_\.\a-zA-Z0-9]{1,61}[a-zA-Z0-9]))\.([a-zA-Z]{2,13}|[a-zA-Z0-9-]{2,30}\.[a-zA-Z]{2,3})$"
jump = r.text.replace("<BR>", "\n")
nohtml = re.sub("<.*?>", "", jump)
nohtml = nohtml.replace(" ", "")
matches = re.finditer(regex, nohtml, re.MULTILINE)
for match in matches:
    subdomains.append(match.group())
subdomains = list(set(subdomains[1:]))
time.sleep(0.5)
print(f"Subdomains found with crt.sh:")
time.sleep(1)
for sub in subdomains:
    print(f"[+] {sub}")
    write_to_file(sub)
    time.sleep(0.07)

#####

# Function: Obtains domain certificate information on the
target ,

```

```

# specifically , Issuer name (third party) and common names as a
    POC
# to identify that the correct domain was chosen .

# Performs a 'get' http req for CRT.sh and query parameter and
    appends
# inputted domain onto the end and outputs column 6,7 values
    and returned .
# 'Issuer Name' and 'Common Name' are these columns and are
    also written
# to the same file .

#####

def get_domain_info(domain):

    response = requests.get(f"https://crt.sh/?q=%.{domain}")
    soup = BeautifulSoup(response.text , 'html.parser')
    rows = soup.find_all('tr')

    for row in rows:
        cols = row.find_all('td')
        if len(cols) >= 6:
            common_name = cols[6].text

```

```

        issuer = cols[7].text.replace(", ", " ") #.replace
            removes the default
#commas which would cause issues with the visual
            representation graph.
time.sleep(1)
write_to_file(common_name)
write_to_file(issuer)
print ("")
print ("DOMAIN CERTIFICATE INFORMATION - Issuer
        Name and Common Name:")
print (f"Common Name: {common_name}\nIssuer: {
        issuer}")
return

#####

# This function performs a DNS lookup request for 5 records
    associated with DNS
# the results are stored in a {} array and utilises the socket
    library to perform
# these lookup requests. Then the information is outputted and
    written to the file.
def dns_lookup(domain_name):

```

```

result = {}
result['A'] = socket.gethostbyname(domain_name)
result['AAAA'] = socket.gethostbyname_ex(domain_name)[-1]
result['CNAME'] = socket.gethostbyname_ex(domain_name)[0]
result['NS'] = socket.gethostbyname_ex(domain_name)[1]
result['MX'] = socket.gethostbyname_ex(domain_name)[2]
print ("")
time.sleep(1)
write_to_file(result)
print ("DNS INFORMATION (A, AAAA, CNAME, NS and MX Records)
:")
print ("Domain Name:", domain, "results of:", result)

#####

# The below function enumerates the IP block range of the
    target domain
# inputted utilising the socket library and to split the last
    octet and
# append a subnet of /24 to the end to demonstrate the IP block
    range
# which is then outputted and also written to the file.

```

```

def get_ip_block_range(domain):
    try:
        ip_address = socket.gethostbyname(domain)
        ip_block = ip_address.split(".")[0:3]
        print("")
        time.sleep(1)
        print("IP BLOCK RANGE OF TARGET:")
        subnet = ".".join(ip_block) + ".0/24"
        print(subnet)
        write_to_file(subnet)

    except socket.gaierror:
        return None

# Writing to the text file. Stores all enumerated OSINT
# information
# with each element added having a commas after it to seperate
# ready
# for the node link graph representation
def write_to_file(data):
    with open("OSINT_information.txt", "a") as file:
        file.write(f"{data},")

```



```
#####

#ANALYSIS - Graphing:

# This function generates automatically the node-link graph
    visual
# representation of the given domain name target and all of the
# information enumerated to produce a well rounded informative
# graph to showcase to non-technical professionals. It does so
    by
# reading the written to file and extracting line by line the
# elements and sets them as a node and performs a linkage from
    each
# node to the parent centre node and adds edges as the links
    and saves
# locally and sets a default figure size of 10x10.

def create_jpg_diagram(file_name , parent):
    G = nx.Graph()

    with open(file_name , 'r') as file:
        for line in file:
            elements = line.strip().split(',')
            for element in elements:
```

```

        G.add_node(element)

        if element != parent:
            G.add_edge(parent, element)

plt.figure(figsize=(10,10))
nx.draw(G, with_labels=True)
plt.savefig('OSINT_diagram.jpg')

print("")
print("VISUAL REPRESENTATION:")
print("Analysis Image Created, see current directory!")

# User input for the target domain to OSINT scrape and
    enumerate.

domain = input("Enter a domain to start the process (please
    exclude prefixes i.e 'www.'): ")

#Function callouts
get_subdomains(domain)
get_domain_info(domain)
dns_lookup(domain)
get_ip_block_range(domain)

# Pre-set values.

```

```
file_path = 'OSINT_information.txt '  
image_path = 'OSINT_Data_diagram.jpg '  
create_jpg_diagram("OSINT_information.txt", domain)
```

```
# Cameron Noakes
```

A.2 APPENDIX B

This appendix A details the full code program of the DNSDETECT novel solution which can be seen below.

```
import socket
import boto3

# Enumerating IPv4 address of domain
def get_ips_from_dns(domain_name):
    try:
        # Use the socket library to perform a DNS lookup on the
        # domain_name
        answers = socket.getaddrinfo(domain_name, None)

        # Extract the IPv4 addresses from the answers
        ips = [answer[4][0] for answer in answers if answer[0]
                == socket.AF_INET]

        # Return the unique IPv4 addresses from ips variable
        return list(set(ips))

    # Exception handling of errors.
    except Exception as e:
        print("Error performing DNS lookup:", e)
```

```

        return []

# Enumerating the IP block range
def generate_ip_block(ips):
    ip_block = []
    # Splits the IP per '.' and generates all 0-255 host
    outcomes.
    for ip in ips:
        ip_parts = ip.split(".")
        for i in range(1, 256):
            new_ip = ".".join(ip_parts[:-1] + [str(i)])
            ip_block.append(new_ip)
    return ip_block

def lambda_handler(event, context):

    # Enter the domain name
    domain_name = "example.com"

    # Connecting to the SNS service created.
    client = boto3.client(
        "sns",
        aws_access_key_id = "REDACTED",

```

```

        aws_secret_access_key = "REDACTED",
        region_name = "us-east-2"
    )

# Calls the intended functions.
ips = get_ips_from_dns(domain_name)
ip_block = generate_ip_block(ips)

# Test Objectives ###
# --- remove comment below to make 'A' record outside range
# ---
#ips = ['87.34.24.112']

# Checks if ips (A record) is in IP block range ,
# if so, compromise detected , alert recieved.
found = False
for ip in ips:
    if ip in ip_block:
        found = True
        break
    break

if not found:
    client.publish(

```

```

TopicArn = "arn:aws:sns:us-east-2:REDACTED:
    dns_tooling",
Message = "#####\n" +
    domain_name + "\n#####\n\
nAWS SNS Issued Message: \nA target domain has
been entered and has detected that the DNS 'A'
record is outside of the company's IP block range
!\n\n\n####\nDATA:\n####\n" + "IP address A record
: "+ str(ips) + "with IP block range of: " + str(
ip_block) + "\n\n"
)
# Cameron Noakes

```

References

- [1] A. Kothia, B. Swar, F. J. (2019). Knowledge extraction and integration for information gathering in penetration testing. *2019 IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 19, 330–335.
- [2] A. Kowta, K. Bhowmick, J. K. N. J. (2021). Analysis and overview of information gathering tools for pentesting. *International Conference on Computer Communication and Informatics*, (pp. 1–13).
- [3] A. Santhosh, R. K. (2021). Identifying subdomains of the website using sublist3r and comparing sublist3r amass, knockpy. *Proceedings of the National Conference on Emerging Computer Applications (NCECA)*, (pp. 230).
- [4] A. Wiradarma, S. S. (2019). It risk management based on iso 31000 and owasp framework using osint at the information gathering stage (case study: X company). *International Journal of Computer Network and Information Security*, (pp. 17–29).
- [5] Alex, V. (2022). A reconnaissance based automatic tool for origin server discovery through misconfigured web servers.
- [6] Anton, S. D. D., Fraunholz, D., & Schneider, D. (2020). Investigating the ecosystem of offensive information security tools. *arXiv preprint arXiv:2012.08811*.
- [7] C. Carpena, A. W. (2014). A survey of ipv6 address usage in the public domain name system.
- [8] C. Marques, S. Malta, J. M. (2021). Dns dataset for malicious domains detection. *Data in Brief*, 38, 107342.
- [9] D. Liu, S. Hao, H. W. (2016). All your dns records point to us. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, (pp. 1414–1425).
- [10] Degani, L., Bergadano, F., Mirheidari, S. A., Martinelli, F., & Crispo, B. (2022). Generative adversarial networks for subdomain enumeration. *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, (pp. 1636–1645).
- [11] Garrett (2005). Ajax: A new approach to web applications.

- [12] H. Alkilani, A. Q. (2021). Osint techniques integration with risk assessment iso/iec 27001. *International Conference on Data Science, E-learning and Information Systems*, (pp. 1–5).
- [13] I. Vacas, I. Medeiros, N. N. (2018). Detecting network threats using osint knowledge-based ids. *European Dependable Computing Conference (EDCC)*, (pp. 1–8).
- [14] K. Baig, K. K. (2021). Passive enumeration in web application security. *International Journal of Modern Agriculture*, 10(2), 804–812.
- [15] K. Wang, S. B. (2016). Building a local passive dns capability for malware incident response. *Virus bulletin conference*.
- [16] M. Yamin, M. Ullah, H. U. B. K. M. H. (2022). Mapping tools for open source intelligence with cyber kill chain for adversarial aware security. *Mathematics*, 10(12).
- [17] Mataracioglu, T. & Ozkan, S. (2011). Governing information security in conjunction with cobit and iso 27001. *arXiv preprint arXiv:1108.2150*.
- [18] Mitropoulos, Louridas, P. K. (2019). Defending against web application attacks: Approaches, challenges and implications. *IEEE Transactions on Dependable and Secure Computing*, 2(16), 188–203.
- [19] P. Kashyap, V. S. (2021). Analysis of different methods of reconnaissance. *Integrated Intelligent Computing Communication Security (ICIIC 2021)*, 4(3).
- [20] Pratama, W. (2019). Open source intelligence testing using the owasp version 4 framework at the information gathering stage (case study: X company). *Computer Network and Information Security*, (pp. 8–12).
- [21] Rai, B. K., Verma, R., & Tiwari, S. (2021). Using open source intelligence as a tool for reliable web searching. *SN Computer Science*, 2(5), 402.
- [22] Ramadhan, R. (2020). Sudomy: Information gathering tools for subdomain enumeration and analysis. *IOP Conference Series: Materials Science and Engineering*, (pp. 1–9).
- [23] S. Gong, J. Cho, C. L. (2018). A reliability comparison method for osint validity analysis. *IEEE Transactions on Industrial Informatics*, 14(12), 5428–5435.
- [24] S. Sharma, K. S. (2022). Expedite the process of reconnaissance: Eagle’s eye of security. *International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS)*, 1, 1–5.
- [25] Susanto, H., Almunawar, M. N., & Tuan, Y. C. (2012). Information security challenge and breaches: novelty approach on measuring iso 27001 readiness level. *International Journal of Engineering and Technology*, 2(1), 67–75.

- [26] Thorsell, G. (2022). Dns enumeration techniques and characterizing dns vulnerabilities.
- [27] U. Dar, A. I. (2018). The silent art of reconnaissance: The other side of the hill. *International Journal of Computer Networks and Communications Security*, 6(12), 250–263.
- [28] V. Redrowthu, I. Ahmed, S. R. S. A. V. R. S. R. (2022). Automation of recon process for ethical hackers. *International Conference for Advancement in Technology (ICONAT)*, (pp. 1–7).
- [29] W. Allen, A. Gerals, A. M. (2005). Automated detection of malicious reconnaissance to enhance network security. *Proceedings. IEEE SoutheastCon, 2005.*, (pp. 450–454).
- [30] Yeboah-Ofori, A. & Brimicombe, A. (2018). Cyber intelligence and osint: developing mitigation techniques against cybercrime threats on social media. *International Journal of Cyber-Security and Digital Forensics (IJCSDf)*, 7(1), 87–98.
- [31] Zoder, O. (2020). Automated collection of open source intelligence.